

Writing Logbook — Assignment 05

CSP529 Technical Writing & Publishing

Reading Research for Evidence: Constructing an Evidence Matrix

MT26MCS024 — Gaurav Acharya MT26MCS032 — Sandeep

September 2026

1. Assignment Overview

Item	Detail
Practical Title	Practical 5 — Reading Research for Evidence: Constructing an Evidence Matrix
Group Topic	Operating System Process / Scheduling (Group 5)
Focused Question	How do different CPU scheduling approaches affect waiting time, response time, throughput, and fairness under different workload characteristics?
Group Members	Gaurav Acharya (MT26MCS024), Sandeep (MT26MCS032)
Guide	Dr. A. S. Mokhade
Mode	In-class close reading and group analysis (2 hours)
Date of Completion	September 2026

2. Objectives & Learning Outcomes

This practical moved from *finding and citing sources* to *reading research literature systematically*. The key principle was: “Do not read a research paper merely to know what it says. Read it to determine what evidence it provides for the question you are investigating.” The learning outcomes included:

- Reading a research paper with a specific technical question in mind.
- Identifying the problem, method, evidence, findings, conditions, and limitations of each study.
- Distinguishing a paper’s general contribution from the evidence relevant to a particular question.
- Tracing extracted information to its exact location in the paper (page, section, table, figure).
- Comparing evidence from multiple research papers systematically.
- Constructing an Evidence Matrix that can support technical synthesis and literature-review writing.

3. Work Performed — Stage by Stage

3.1. Stage 1: Begin With the Question

Before reading any papers, both group members discussed the assigned focused question:

“How do different CPU scheduling approaches affect waiting time, response time, throughput, and fairness under different workload characteristics?”

Understanding the Question

The question requires evaluating and comparing various CPU scheduling algorithms (FCFS, SJF, Round Robin, Multilevel Queue, etc.) and determining how they impact key system performance metrics. Specifically, we needed to understand how these outcomes shift when the type of workload changes — from CPU-bound to I/O-bound tasks, or from batch processing to interactive systems.

Key Terms Clarified

- **Waiting Time:** The total amount of time a process spends waiting in the ready queue.
- **Response Time:** The time elapsed from when a request is submitted until the first response is produced (crucial for interactive systems).
- **Throughput:** The number of processes that complete execution per time unit.
- **Fairness:** Ensuring no process suffers from starvation and that CPU time is distributed equitably among processes of similar priority.
- **Workload Characteristics:** The nature of processes being executed — burst times, arrival rates, and whether they are CPU-intensive or I/O-intensive.

3.2. Stage 2: Read Your Paper for Evidence

Each group member read one research paper with the focused question continuously in mind, using annotations (Q, M, E, F, C, L).

Paper 1 (Sandeep — MT26MCS032)

Title: “Comparative Analysis of CPU Scheduling Algorithms for Performance Efficiency”

- **Problem:** Evaluates the efficiency of various CPU scheduling algorithms to determine how the choice of strategy influences system responsiveness, throughput, waiting time, and turnaround time.
- **Method:** Simulated a uniprocessor framework to calculate and compare performance using Gantt charts for FCFS, SJF, SRTF, RR, Priority, HRRN, and FKS algorithms.
- **Data/Experimental Setting:** A single dataset featuring five processes (P1–P5) with predetermined burst times and priorities, assuming all arrive at time zero.
- **Relevant Finding:** SRTF achieved the lowest average turnaround time (34.2ms) and waiting time (19.2ms); Round Robin recorded the highest (turnaround: 51ms, waiting: 36ms).
- **Evidence:** Table 3 (Average Turnaround Time and Waiting Time) and Gantt charts (Figures 1 and 2).
- **Conditions/Assumptions:** All jobs ready at time zero, burst times known in advance, processes run to completion without I/O interruptions, negligible context-switching time.
- **Limitations:** SRTF and FKS rely on the unrealistic assumption of knowing exact future burst times. The study’s zero context-switching overhead assumption ignores the primary drawback of preemptive algorithms.
- **Contribution:** Preemptive algorithms like SRTF minimize waiting times for predictable workloads (though with unrealistic assumptions). Round Robin, while fair for interactive systems, can severely degrade waiting time if the time quantum is sized incorrectly.

Paper 2 (Gaurav Acharya — MT26MCS024)

Title: “Comparative Analysis of Different CPU Scheduling Algorithms with Respect to Average Waiting Time and Average Turnaround Time”

- **Problem:** Analyzes the relationship between existing CPU scheduling algorithms (FCFS, SJF, RR, Priority Based) with respect to AWT and ATAT to identify the fastest algorithm.
- **Method:** Designed five mathematical cases by altering CPU burst times (adding 1, subtracting 2, etc.) and calculated AWT and ATAT for each algorithm across all cases.
- **Data/Experimental Setting:** A ready queue of four processes (P1–P4) with varying CPU burst times and priorities, tested across 5 rigidly formulated cases.
- **Relevant Finding:** SJF consistently produced the lowest AWT and ATAT across all five test cases. CPU burst time was found to be directly proportional to average waiting and turnaround times.
- **Evidence:** Tables 8–11 and 3D line graphs (Figures 1–4) for each algorithm.
- **Conditions/Assumptions:** Findings apply strictly to the five formulated cases where CPU burst times scale linearly.
- **Limitations:** Only evaluates waiting time and turnaround time — completely ignores response time, throughput, and fairness.
- **Contribution:** SJF provides the absolute lowest waiting time regardless of how burst times scale mathematically. However, the paper cannot explain how these workloads behave in interactive, real-world environments.

3.3. Stage 4: Constructing the Evidence Matrix

We constructed a group Evidence Matrix comparing both papers across eight dimensions:

Dimension	Paper 1	Paper 2
Problem addressed	Evaluated scheduling algorithm efficiency based on waiting time and turnaround time	Found the relationship between algorithms regarding AWT and ATAT; identified the fastest algorithm
Method	Simulated uniprocessor framework with Gantt charts for 7 algorithms (FCFS, SJF, SRTF, RR, Priority, HRRN, FKS)	Designed 5 cases by mathematically altering burst times; tested FCFS, SJF, RR, and Priority
Data setting	5 processes (P1–P5) with known burst times, all arriving at time zero	4 processes (P1–P4) with varying burst times across 5 formulated cases
Main finding	SRTF achieved lowest AWT (19.2 ms) and ATAT (34.2 ms)	SJF consistently had lowest AWT and ATAT across all 5 cases
Evidence	Table 3 comparing metrics; Gantt charts (Figures 1–2)	Tables 8–11 and 3D line graphs (Figures 1–4)
Conditions	Burst times known in advance, zero arrival time, negligible context-switching	Evaluated under specific mathematical modifications to burst times
Limitations	SRTF depends on unrealistic future burst estimates; zero context-switching assumed	Did not measure response time, throughput, or fairness

Dimension	Paper 1	Paper 2
Contribution to question	Proves SRTF/SJF minimizes waiting for batch; RR better for interactive fairness	Proves burst time is directly proportional to waiting time; SJF minimizes regardless of burst variations

3.4. Stage 5: Looking Across the Matrix

Without attempting full synthesis, we examined the completed matrix:

1. **Comparable dimensions:** We can directly compare the methodologies and evaluation metrics. Both papers evaluate FCFS, SJF, and Round Robin by calculating Average Waiting Time (AWT) and Average Turnaround Time (ATAT).
2. **Similar evidence:** Both papers provide identical conclusions regarding SJF minimizing wait times. Paper 1 found SRTF (preemptive SJF) yielded the lowest waiting and turnaround times. Paper 2 verified that SJF consistently provided the lowest AWT across all 5 experimental cases.
3. **Apparent differences:** The experimental setups differ significantly. Paper 1 used a single static dataset of 5 processes to evaluate 7 algorithms. Paper 2 used 4 processes but actively modified their burst times mathematically across 5 cases to observe scaling behaviour.
4. **Missing information:** Our focused question explicitly asks about “response time,” “throughput,” and “fairness.” However, **both papers focused almost exclusively on waiting time and turnaround time**, failing to provide hard quantitative data for measuring fairness (starvation rates) or interactive response times.

4. Individual Reflections

4.1. Sandeep (MT26MCS032 — Paper 1)

- **E1 — Most useful evidence:** Table 3 (Average Turnaround Time and Average Waiting Time) provides direct, quantitative evidence comparing eight algorithms, proving that Round Robin yields the highest waiting time (36 ms) compared to SRTF (19.2 ms) for this specific workload.
- **E2 — What the paper could not answer:** The paper briefly notes that Round Robin promotes fairness while Priority scheduling can cause starvation, but provides *no numerical evidence or graphs* to actually measure fairness under different workloads.
- **E3 — Value of manual reading:** The abstract simply states the study evaluates efficiency using a simulation framework. Reading the full paper revealed massive limitations: it assumed zero context-switching time and that all processes arrive at exactly time zero, which heavily skews real-world applicability.

4.2. Gaurav Acharya (MT26MCS024 — Paper 2)

- **E1 — Most useful evidence:** The Conclusion section and Tables 8–11 synthesize the results of five cases to prove mathematically that CPU burst time is strictly and directly proportional to both AWT and ATAT.
- **E2 — What the paper could not answer:** The paper did not address throughput, response time, or fairness at all — it was strictly a mathematical analysis of AWT and ATAT for four specific algorithms.
- **E3 — Value of manual reading:** The abstract states that five cases were designed to find the relationship between algorithms. By reading Section 4, it became clear these

were not random workloads but highly specific mathematical modifications (adding 1, subtracting 2) designed specifically to force linear scaling behaviour.

5. Writing Logbook Entries

1. **Most difficult to extract:** Quantitative comparative results were buried in dense tables and 3D graphs rather than stated explicitly in text. Extracting specific numbers relevant to our focused question required careful graph reading and cross-referencing across multiple tables.
2. **Most useful section/table/figure:** Paper 1's Table 3 (average turnaround and waiting times across 8 algorithms) and Paper 2's Tables 8–11 (AWT and ATAT across 5 cases) provided the most directly useful evidence.
3. **What the abstracts failed to tell:** Paper 1's abstract did not mention its zero context-switching assumption. Paper 2's abstract did not reveal that its “five cases” were artificial mathematical modifications, not realistic workload scenarios.
4. **Difference of reading with a focused question:** Reading with the question “How do scheduling approaches affect waiting time, response time, throughput, and fairness?” immediately revealed a critical gap — **neither paper measured response time, throughput, or fairness quantitatively**, despite both claiming to evaluate “scheduling algorithm efficiency.”
5. **Further investigation needed:** We need papers that specifically measure response time and fairness under dynamic workloads, and papers that evaluate scheduling in real operating systems rather than only simulations with known burst times.

6. Challenges Encountered

1. **No AI tools allowed:** Manual reading and annotation was significantly slower than AI-assisted summarization, but forced deeper engagement with the methodology sections that AI summaries skip.
2. **Evidence traceability:** Consistently recording exact table/figure numbers for every extracted finding required discipline — we had to go back and re-check locations multiple times.
3. **Gap identification:** Discovering that both selected papers failed to address 3 out of 4 metrics in our focused question (response time, throughput, fairness) was the most important finding but also the most uncomfortable — it meant our evidence matrix had fundamental gaps.
4. **Matrix alignment:** The two papers used different numbers of processes (5 vs. 4), different sets of algorithms (7 vs. 4), and different experimental designs (static vs. scaled), making direct comparison require careful normalization of terminology.
5. **Distinguishing claims from data:** Paper 2 claimed mathematical proportionality between burst time and waiting time, but the “proof” relied on only 5 hand-constructed cases — we had to recognize this as a limited observation, not a general proof.

7. Key Takeaways

1. Reading with a focused question is transformative — it converts passive reading into active evidence hunting and immediately reveals what papers *cannot* answer.
2. An Evidence Matrix forces systematic, side-by-side comparison that exposes both agreements (both papers confirm SJF minimizes AWT) and gaps (neither measures fairness quantitatively).
3. Evidence must be traceable to exact locations — “the paper says SJF is best” is insufficient; “Table 3 shows SRTF achieved 19.2ms AWT” is what a literature review requires.

4. Manual reading reveals critical methodological assumptions (zero context-switching, known burst times, all-at-once arrival) that abstracts and AI summaries consistently omit.
5. The most important finding from this practical was the **absence of evidence**: our focused question asked about 4 metrics, but both papers only addressed 2 — identifying this gap is itself a research contribution.
6. The TWP practical progression is now clear: *What am I saying?* (Practical 3) → *Can I trust the source?* (Practical 4) → *What evidence does the paper provide for my question?* (Practical 5).